

Lab Assignment

C17 — Introduction to Template Functions

Topics: template • typename • Type Deduction • Generic Programming • Operator Constraints

1. Objective

This lab introduces **template functions** — one of C++'s most powerful tools for writing code that works correctly with *any* data type without duplication. Rather than writing separate versions of the same function for `int`, `double`, and `string`, a single template definition lets the compiler generate the correct version automatically.

By the end of this lab you will be able to:

- ▶ Explain the problem that templates solve and why function overloading alone is insufficient.
- ▶ Write a template function using the `template <typename T>` syntax.
- ▶ Understand how the compiler deduces the type `T` from the arguments at each call site.
- ▶ Identify the implicit constraint that `T` must support the operators used inside the template.
- ▶ Apply templates to `max2()`, `min2()`, `swap2()`, and `printArray()`.
- ▶ Connect template functions to the standard library — `std::max`, `std::min`, `std::swap` are templates too.

2. Background

2.1 The Repetition Problem

Suppose you need a function that returns the larger of two values. Without templates you must write one version per type:

```
// Three functions — identical bodies, only the type differs
int  maxInt  (int a,   int b)   { return (a > b) ? a : b; }
double maxDouble(double a, double b) { return (a > b) ? a : b; }
string maxString(string a, string b) { return (a > b) ? a : b; }
```

This violates the **DRY principle** (Don't Repeat Yourself). If the logic ever needs to change, every copy must be updated. Templates eliminate this duplication by letting the type itself become a parameter.

2.2 Template Syntax

A template function is declared with a **template header** immediately before the function:

```
template <typename T>    // T is a placeholder — the compiler fills it in
T max2(T a, T b) {
    return (a > b) ? a : b;
}
```

Element	Meaning
template	Keyword that begins every template definition.
<typename T>	Declares T as a type parameter. The name T is conventional but any identifier works.
T max2(T a, T b)	T is used as the return type and both parameter types — all three must match.
(a > b)	Implicit constraint: T must support operator>. The compiler checks this when T is substituted.

2.3 Type Deduction — No Angle Brackets Needed

The compiler infers `T` automatically from the argument types at each call site. You do not need to specify it manually:

```
max2(45, 10)           // compiler deduces T = int
max2(3.85, 2.13)       // compiler deduces T = double
max2("Homer", "Marge") // compiler deduces T = string

// You CAN specify T explicitly — useful when deduction is ambiguous:
max2<double>(10, 3.85) // forces T = double even though 10 is int
```

2.4 Implicit Operator Constraints

Every template function places an **implicit constraint** on `T`: `T` must support every operator used inside the function body.

- ▶ `max2()` and `min2()` require `operator>` and `operator<`.
- ▶ `swap2()` requires copy assignment `operator=` and a copy constructor.
- ▶ `printArray()` requires `operator<<` for stream output.

If you pass a type that does not satisfy the constraint, the compiler reports an error *at the call site* — not inside the template definition itself.

2.5 Connection to the Standard Library

Once you understand templates you will recognize that the STL functions you have already used are all templates:

```
std::max(a, b)           // same idea as your max2()
std::min(a, b)           // same idea as your min2()
std::swap(a, b)          // same idea as your swap2()
std::sort(arr, arr+n)    // works on any array whose elements support operator<
std::vector<int>         // vector itself is a class template
```

3. Required Experiments

Implement all four template functions in a header file named `templates.h`. Write one experiment function per template and call all four from `main()`.

Experiment 01 — `max2()` and `min2()`

Write template functions `max2()` and `min2()` that each accept two arguments of the same type and return the larger or smaller value.

Signature:

```
template <typename T>
T max2(T a, T b);

template <typename T>
T min2(T a, T b);
```

Test with all three types:

Type	Values	Expected max2	Expected min2
int	Homer age 45, Bart age 10	45	10
double	Lisa GPA 3.85, Bart GPA 2.13	3.85	2.13
string	"Homer", "Marge"	"Marge"	"Homer"

```
void experiment01() {
    cout << "=== Experiment 01: max2() and min2() ===\n\n";

    // int
    int a = 45, b = 10;
    cout << "Homer age: " << a << " Bart age: " << b << "\n";
    cout << " max2 -> " << max2(a, b) << "\n";
    cout << " min2 -> " << min2(a, b) << "\n\n";

    // double
    double g1 = 3.85, g2 = 2.13;
    cout << "Lisa GPA: " << g1 << " Bart GPA: " << g2 << "\n";
    cout << " max2 -> " << max2(g1, g2) << "\n";
    cout << " min2 -> " << min2(g1, g2) << "\n\n";

    // string — compared alphabetically
    string s1 = "Homer", s2 = "Marge";
    cout << "Names: " << s1 << " " << s2 << "\n";
    cout << " max2 -> " << max2(s1, s2) << "\n";
    cout << " min2 -> " << min2(s1, s2) << "\n";
}
```

Experiment 02 — `swap2()`

Write a template function `swap2()` that exchanges the values of two variables of the same type. Parameters must be passed by **reference** — passing by value would swap copies and leave the originals unchanged.

Signature:

```
template <typename T>
void swap2(T& a, T& b);    // T& required — must modify the originals
```

Test cases:

- Swap two `int` values (Homer's age and Bart's age).
- Swap two `double` values (Lisa's GPA and Bart's GPA).
- Swap two `string` values (Homer's name and Marge's name).

Verify:

Print both variables *before* and *after* calling `swap2()` to confirm the exchange.

```
void experiment02() {
    cout << "=== Experiment 02: swap2() ===\n\n";

    // int
    int age1 = 45, age2 = 10;
    cout << "Before: Homer=" << age1 << "   Bart=" << age2 << "\n";
    swap2(age1, age2);
    cout << "After : Homer=" << age1 << "   Bart=" << age2 << "\n\n";

    // double
    double gpa1 = 3.85, gpa2 = 2.13;
    cout << "Before: Lisa=" << gpa1 << "   Bart=" << gpa2 << "\n";
    swap2(gpa1, gpa2);
    cout << "After : Lisa=" << gpa1 << "   Bart=" << gpa2 << "\n\n";

    // string
    string name1 = "Homer", name2 = "Marge";
    cout << "Before: " << name1 << "   " << name2 << "\n";
    swap2(name1, name2);
    cout << "After : " << name1 << "   " << name2 << "\n";
}
```

Experiment 03 — printArray()

Write a template function `printArray()` that prints every element of an array of any type. This experiment demonstrates that templates work with arrays and multiple parameters of different kinds — `T` for the element type and `int` for the size.

Signature:

```
template <typename T>
void printArray(const T arr[], int size, const string& label = "");
```

Test with three arrays:

- `int ages[] = { 45, 42, 10, 8, 38 }`
- `double gpas[] = { 3.85, 2.13, 1.50, 3.92 }`
- `string names[] = { "Homer", "Marge", "Bart", "Lisa", "Maggie" }`

```

void experiment03() {
    cout << "=== Experiment 03: printArray() ===\n\n";

    int    ages[]  = { 45, 42, 10, 8, 38 };
    double gpas[]  = { 3.85, 2.13, 1.50, 3.92 };
    string names[] = { "Homer", "Marge", "Bart", "Lisa", "Maggie" };

    printArray(ages, 5, "Ages ");
    printArray(gpas, 4, "GPAs ");
    printArray(names, 5, "Names");
}

```

Experiment 04 — Explicit Type Specification

In most cases the compiler deduces `T` automatically. This experiment shows when and why you might want to specify `T` explicitly using angle brackets.

Tasks to verify:

- ▶ Call `max2(10, 3.85)` without specifying `T` — observe the compiler error (mismatched types).
- ▶ Call `max2<double>(10, 3.85)` with explicit `T = double` — observe that it now compiles and returns 10.0.
- ▶ Call `max2<int>(10, 3.85)` — observe that 3.85 is truncated to 3.

Key teaching/learning point:

The compiler generates a *separate* compiled version of the template for each distinct type it encounters. `max2<int>`, `max2<double>`, and `max2<string>` are three distinct compiled functions — the template is a blueprint, not the function itself.

```

void experiment04() {
    cout << "=== Experiment 04: Explicit Type Specification ===\n\n";

    // max2(10, 3.85) -- COMPILE ERROR: deduction fails (int vs double)
    // Uncomment the line below to see the error:
    // cout << max2(10, 3.85) << "\n";

    // Explicit T = double: int 10 is promoted to double 10.0
    cout << "max2<double>(10, 3.85) = " << max2<double>(10, 3.85) << "\n";

    // Explicit T = int: double 3.85 is truncated to int 3
    cout << "max2<int>(10, 3.85)      = " << max2<int>(10, 3.85)      << "\n";
}

```

4. Starter Code

Create `templates.h` and implement all four functions. The signatures below are your starting point — replace each comment with the correct body.

```

// templates.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

template <typename T>
T max2(T a, T b) {
    // TODO: return the larger of a and b
}

template <typename T>
T min2(T a, T b) {
    // TODO: return the smaller of a and b
}

template <typename T>
void swap2(T& a, T& b) {
    // TODO: exchange the values of a and b
    // Hint: you need a temporary variable of type T
}

template <typename T>
void printArray(const T arr[], int size, const string& label = "") {
    // TODO: print label (if not empty), then all elements
    // Format: Ages : [ 45, 42, 10, 8, 38 ]
}

```

5. Expected Output

```

=== Experiment 01: max2() and min2() ===

```

```

Homer age: 45   Bart age: 10
  max2 -> 45
  min2 -> 10

```

```

Lisa GPA: 3.85  Bart GPA: 2.13
  max2 -> 3.85
  min2 -> 2.13

```

```

Names: Homer  Marge
  max2 -> Marge
  min2 -> Homer

```

```

=== Experiment 02: swap2() ===

```

```

Before: Homer=45  Bart=10
After  : Homer=10  Bart=45

```

```

Before: Lisa=3.85  Bart=2.13
After  : Lisa=2.13  Bart=3.85

```

```

Before: Homer  Marge
After  : Marge  Homer

```

```

=== Experiment 03: printArray() ===

```

```
Ages : [ 45, 42, 10, 8, 38 ]
GPAs : [ 3.85, 2.13, 1.5, 3.92 ]
Names: [ Homer, Marge, Bart, Lisa, Maggie ]

=== Experiment 04: Explicit Type Specification ===

max2<double>(10, 3.85) = 10
max2<int>(10, 3.85)   = 10
```

6. Common Mistakes

Mistakes to avoid

- ▶ **swap2 by value — nothing is exchanged:** `void swap2(T a, T b)` swaps copies. Use `T& a`, `T& b` to modify the originals.
- ▶ **Mixing types without explicit T:** `max2(10, 3.85)` deduces `T = int` from the first arg and `T = double` from the second — a conflict. Fix with `max2<double>(10, 3.85)`.
- ▶ **Defining the template in a .cpp file:** The compiler needs the full template definition when it instantiates it. Keep template functions in a header file (`.h`), not a separate `.cpp`.
- ▶ **Passing an unsupported type:** Passing a custom class that does not overload `operator>` to `max2()` gives a compile error. The constraint is implicit — it is your responsibility to ensure `T` supports the required operators.

7. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. What problem do template functions solve that function overloading alone cannot?
- ▶ 2. What does the compiler actually do when it sees `max2(45, 10)`? What is generated?
- ▶ 3. Why must `swap2()` take its parameters by reference? What happens if it takes them by value?
- ▶ 4. What is an implicit constraint in a template function? Give an example using `max2()`.
- ▶ 5. Why must template functions be defined in header files rather than in separate `.cpp` files?

Key idea

A template function is a **blueprint**, not a function. The compiler reads it when you write it, but only *generates* actual machine code when it sees a call with a concrete type. `max2<int>`, `max2<double>`, and `max2<string>` are three separate compiled functions — all produced from the same one-line template body.